Machine Learning

Módulo 2

EDUCACIÓN IT

Clases y objetos

Funciones

En un lenguaje de programación, una **función** es un bloque de código que se ejecuta únicamente cuando es llamado.

- Con la sentencia «def» se establece que lo que sigue, es la definición de una función.
- Le sigue el nombre de la función.
- Y los parámetros, que son opcionales.
- Notar la importancia de la indentación en Python.

```
def miFuncion(n):
    if n == 0:
        print("Igual a cero")
    elif n > 0:
        print("Mayor a cero")
    else:
        print("Menor a cero")
```

Funciones

- Los parámetros pueden tener valores por defecto.
- Una función por lo general devuelve un valor, eso se establece con la palabra «return».
- El valor devuelto puede ser más de uno, y diferentes tipos de datos y objetos.
- Se puede asignar a una variable, el resultado de retorno de una función.

```
def dividir(dividendo, divisor = 1):
    if divisor == 0:
        return "No se puede dividir por cero"
    else:
        return dividendo / divisor
```

```
div = dividir(2,0)
print(div)
```

```
No se puede dividir por cero
```

```
div = dividir(277,5)
print(div)
```

```
55.4
```

Funciones

Recursividad

- En ocasiones, según la problemática que se quiera resolver, podemos utilizar la recursividad, que consiste en una función que en su algoritmo, hace referencia a sí misma. Python lo permite admitiendo el uso de las llamadas recursivas.
- Entre las ventajas de usar funciones podemos encontrar que se obtiene código reutilizable y más prolijo o legible.
- Es importante notar también que se puede documentar, mediante la inserción de comentarios y el uso de la función help().

```
def factorial(numero):
    '''
    Devuelve el factorial
    '''
    if numero > 1:
        numero = numero * factorial(numero - 1)
    return numero
```

```
print(factorial(5))
```

```
120
```

```
help(factorial)
```

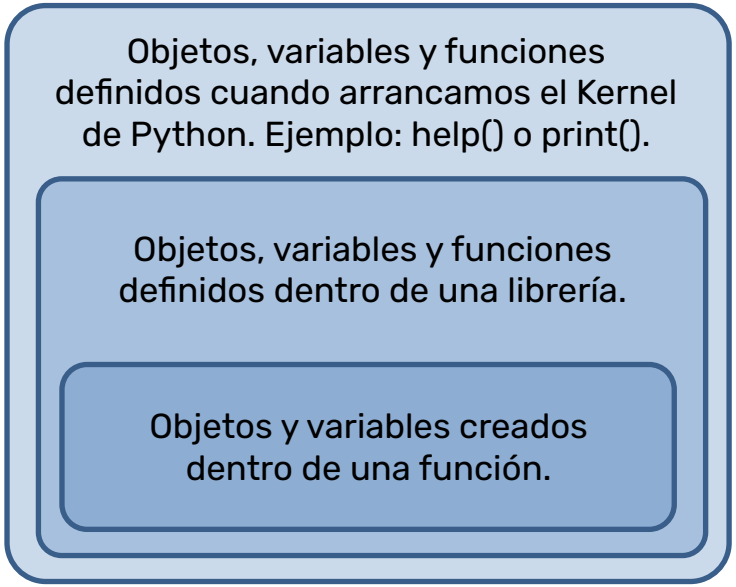
```
Help on function factorial in module __main__:

factorial(numero)
    Devuelve el factorial
```

Funciones

Espacio de nombres

El alcance que se tiene sobre los diferentes objetos se denomina espacio de nombres, dentro de una función puede haber una variable que solo se pueda utilizar dentro del cuerpo de la función, son **variables locales**. Mientras que las variables en el código desde el cual se llama a la función, son **variables globales**.



Funciones - Espacio de nombres

```
def dividir(dividendo, divisor = 1):
    if divisor == 0:
        return "No se puede dividir por cero"
    else:
        return dividendo / divisor
print(dividir(10))
print(divisor)
```

```
10.0
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-158-ddb4ad02885a> in <module>
      5         return dividendo / divisor
      6 print(dividir(10))
----> 7 print(divisor)

NameError: name 'divisor' is not defined
```

En este caso, **divisor** es una variable local dentro de la función. Deja de existir cuando termina el llamado a la función, es justamente lo que denota el mensaje de error.

```
divisor = 5
def dividir(dividendo):
    if divisor == 0:
        return "No se puede dividir por cero"
    else:
        return dividendo / divisor
print(dividir(10))
print(divisor)
```

```
2.0
5
```

En este caso **divisor** es una variable global. En el llamado de la función, como no encuentra una variable local llamada **divisor**, busca una variable global llamada **divisor**.

```
divisor = 5
def dividir(dividendo, divisor = 1):
    if divisor == 0:
        return "No se puede dividir por cero"
    else:
        return dividendo / divisor
print(dividir(10))
print(divisor)
```

```
10.0
5
```

En este caso existe **divisor** como variable global y **divisor** como variable local. El llamado a la función usa siempre primero la variable local.

Funciones

Lambda

La función **Lambda** es una forma conveniente de crear una función en una sola línea. También se las conoce como **funciones anónimas**, ya que no tienen nombre, sino que se asignan a una variable.

```
lambda_producto = lambda x, y: x * y
lambda_producto(4, 5)
```

```
20
```

- Pueden tener cualquier cantidad de argumentos, pero solo una expresión.
- No necesitan un return.
- Muy cómodas para crear funciones rápido.
- Se combinan muy bien con funciones como *map(), apply(), applymap(),* etc.

Python: Clases y objetos

El alto grado de planificación y previsión que requiere la programación es contrario a la propia realidad. El hombre aprende y crea a través de la experimentación, no de la planeación.

La **Orientación a Objetos (POO)** brinda estos métodos de experimentación y logra que los lenguajes sean de más **alto nivel**, es decir, más cercanos a como los humanos pensamos el mundo.

Hasta 1966 la programación fue exclusivamente lineal, hasta que surgieron lenguajes como Simula, SmallTalk, C++, Ada, Delphi o Java...

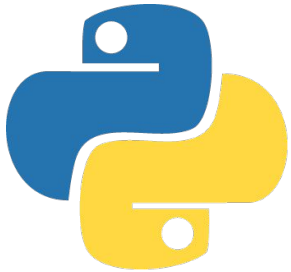
Hoy el lenguaje más popular es Python y su filosofía hace hincapié en la **legibilidad de su código**.

Fue creado a finales de los ochenta. Nació como un proyecto de software libre y posiblemente deba parte de su éxito a la decisión de hacerlo código abierto. Para mediados de los 90, incluyó herramientas de la programación funcional como lambda, reduce, filter y map.

Python: Clases y objetos

La generalización del Big Data en los últimos años, seguida de la explosión de la Inteligencia Artificial, Machine Learning y el surgimiento de la Ciencia de Datos como un nuevo área de trabajo con especialistas propios, ha revolucionado el panorama ya que muchas de las nuevas herramientas que han surgido han sido desarrolladas en Python o nos ofrecen Python como la forma predilecta de interactuar con ellas.

Podemos hablar de tecnología para Big Data como PySpark, de herramientas para Data Science como Pandas, NumPy, Matplotlib o Jupyter. De herramientas del procesamiento del lenguaje natural como NLTK, y por último el área de Deep Learning como Tensorflow, MXNet o Scikit-Learn.



Python: Clases y objetos

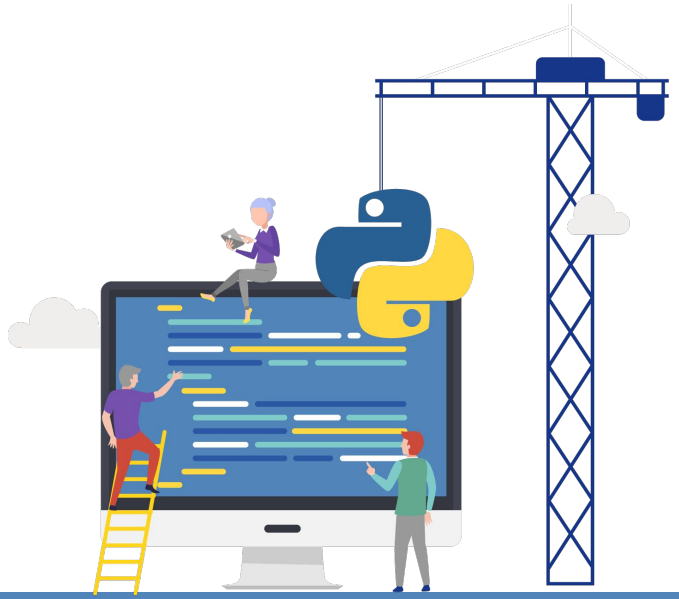
¿Qué es un objeto?

Una estructura de datos y funciones asociadas, llamadas *métodos,* que están agrupados por razones de consistencia y comodidad.

Objeto

Especie = ‘Perro’
Edad = 5
Color = ‘Marron’

Animal_especie = ‘Perro’
Animal_edad = 5
Animal_Color = ‘Marrón’
Correr()
Dormir()



Python: Clases y objetos

¿Qué es una clase?

Es una generalización de un objeto, que oficia de plantilla a partir de la cual se crean los objetos.

Clase Animal

- Especie
- Edad
- Color
- Correr()
- Dormir()

Se instancian (crean) distintos objetos de una clase.

Objeto 1

especie = ‘Perro’
edad = 3
color = ‘Blanco’



Objeto 2

especie = ‘Caballo’
edad = 8
color = ‘Blanco y Negro’





Python: Clases y objetos

- A partir de la sentencia «class» y el nombre de creamos la clase.
- La función '*__init__()*' es el constructor de la clase, esta función se ejecuta cuando se instancia el objeto.

```
class Animal:
    '''
    En esta clase se crean animales
    '''
    def __init__(self, especie, edad, color):
        self.especie = especie
        self.edad = edad
        self.color = color
    def mePresento(self):
        print("Hola, soy", self.especie, ", de color", self.color, "y tengo", self.edad, "años")
    def cumplirAnios(self):
        self.edad = self.edad + 1
```

- La clase posee *atributos* (especie, edad, color) y *métodos* que manipulan esos atributos (mePresento, cumplirAnios).



Python: Clases y objetos

- Creamos los objetos a1 y a2. Al hacerlo se envían los parámetros de inicialización de sus atributos.
- Utilizamos sus métodos para mostrar los atributos y/o modificarlos.
- Este formato de clases, objetos, métodos y parámetros es muy común en Python y lo utilizamos cada vez que invocamos alguna de sus librerías

```
a1 = Animal('Raton', 2, 'Marron')
print(a1.especie)
print(a1.edad)

a2 = Animal('Liebre', 3, 'Gris')
print(a2.especie)
print(a2.edad)
```

```
Raton
2
Liebre
3
```

```
a1.mePresento()
```

```
Hola, soy Raton , de color Marron y tengo 2 años
```

```
a2.mePresento()
```

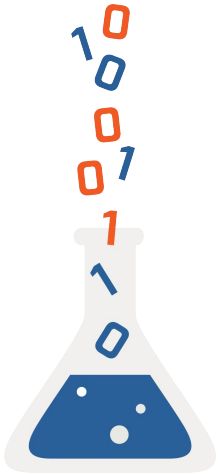
```
Hola, soy Liebre , de color Gris y tengo 3 años
```

```
a1.cumplirAnios()
```

```
a1.mePresento()
```

```
Hola, soy Raton , de color Marron y tengo 3 años
```





Lab Time!

¡Muchas gracias!

¡Sigamos trabajando!